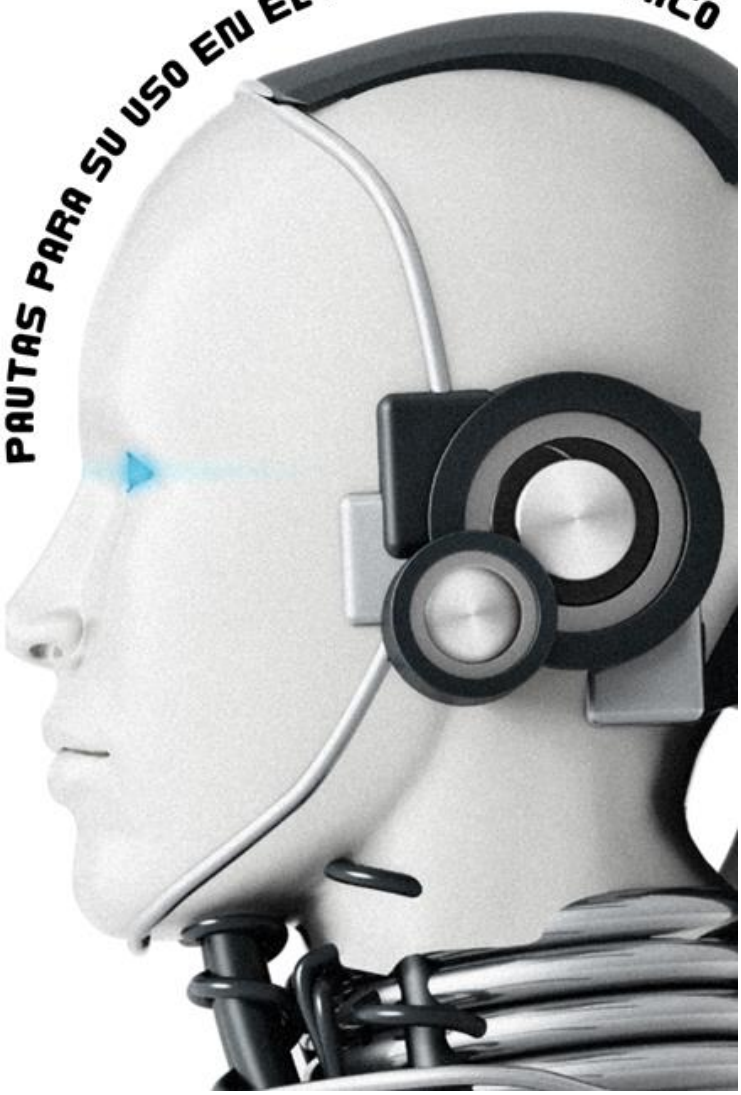
biblioteca
LEOPOLDO MARECHAL

introducción al uso de la

IA

nivel II

PAUTAS PARA SU USO EN EL ÁMBITO ACADÉMICO



Módulo I

Ingeniería de Prompt Avanzada

1. **Técnicas avanzadas de redacción de prompts:**
   - Few-shot learning (aprendizaje de pocos disparos)
   - Chain-of-thought (cadena de pensamientos),
   - Prompt chaining (encadenamiento de indicaciones)
2. **Optimización de prompts con el algoritmo ProTeGi**
3. **Herramientas para la optimización automática de prompts**

Técnicas avanzadas de redacción de prompt

Como vimos anteriormente un prompt es una orden que se ejecuta en un modelo de inteligencia artificial para obtener una respuesta. Sabemos que la calidad de la respuesta depende de nuestra pericia para redactar el prompt correctamente asignando roles y utilizando determinadas palabras para indicar la acción que requerimos que el modelo de IA realice. Sin embargo, existen técnicas avanzadas que nos permiten optimizar los resultados como la cadena de pensamiento (chain-of-thought), aprendizaje con pocos ejemplos (few-shot-learning) o encadenamiento de prompts (Prompt Chaining).

Cadena de pensamiento (chain-of-thought): esta técnica busca que el modelo de IA que estemos usando desglose la orden en varios pasos lo cual nos permite verificar rápidamente errores y permite a la IA comprender mejor la naturaleza de la pregunta.

Ejemplo:

Prompt sin cadena de pensamiento:

Si tengo 5 manzanas y me como 2 ¿Cuántas me quedan?

Repuesta: 3 es correcta, pero… ¿cómo llegó a ese resultado?

Aplicando la cadena de pensamiento la respuesta sería la siguiente:

Si tengo 5 manzanas y me como 2 ¿Cuántas me quedan? Pensá y detallame el paso a paso:

1. Inicialmente tienes 5 manzanas
2. Te comes 2 manzanas

3. Aplicas la operación: 5-2
4. La respuesta es 3

Esto pareciera que no tiene sentido ya que anteriormente había llegado sin dificultad al resultado, sin embargo, observemos el siguiente caso:

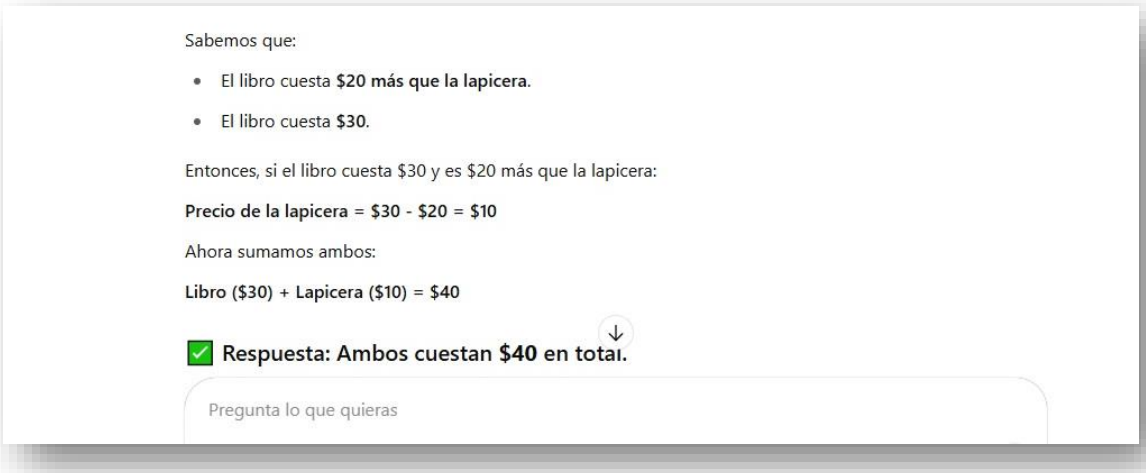
Ejemplo sin cadena de pensamiento:

Un libro cuesta $20 más que una lapicera. Si el libro cuesta $30, ¿Cuánto cuestan ambos juntos?

La respuesta rápida diría que $40 lo cual en principio es correcta sin embargo no muestra el pensamiento lógico.

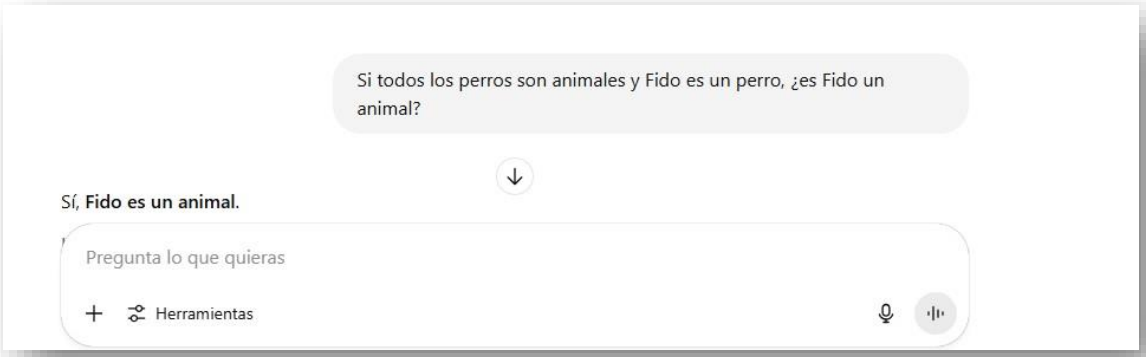
Ejemplo con cadena de pensamiento:

Un libro cuesta $20 más que una lapicera. Si el libro cuesta $30, ¿Cuánto cuestan ambos juntos? Piensa y desglosa un paso a paso:



Ejemplo 2:

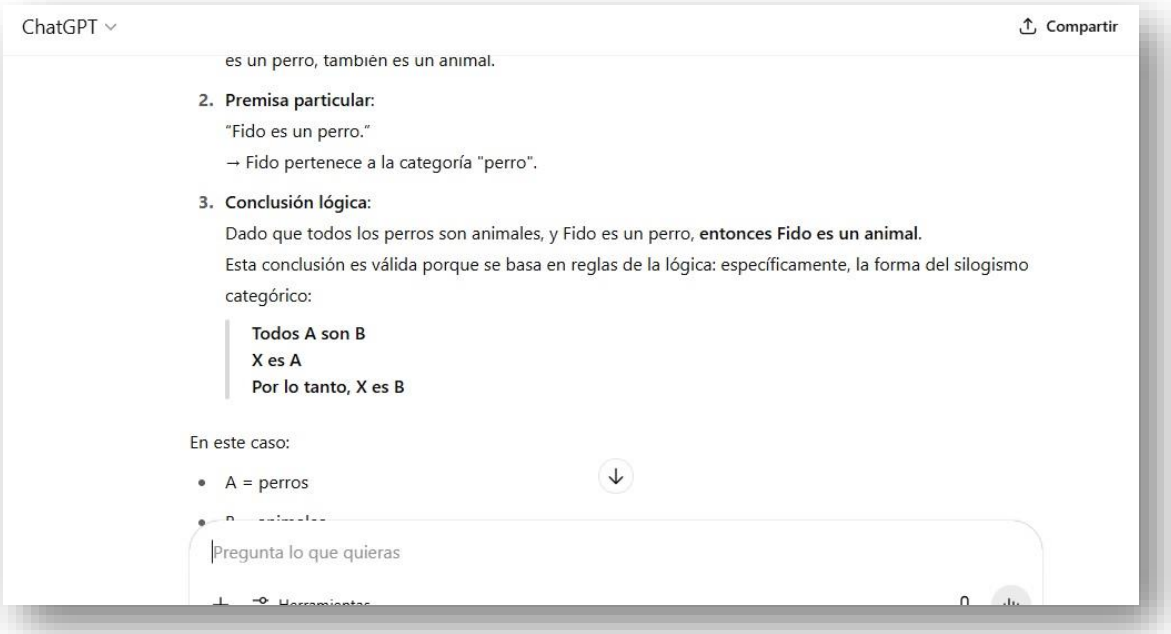
Sin cadena de pensamiento:



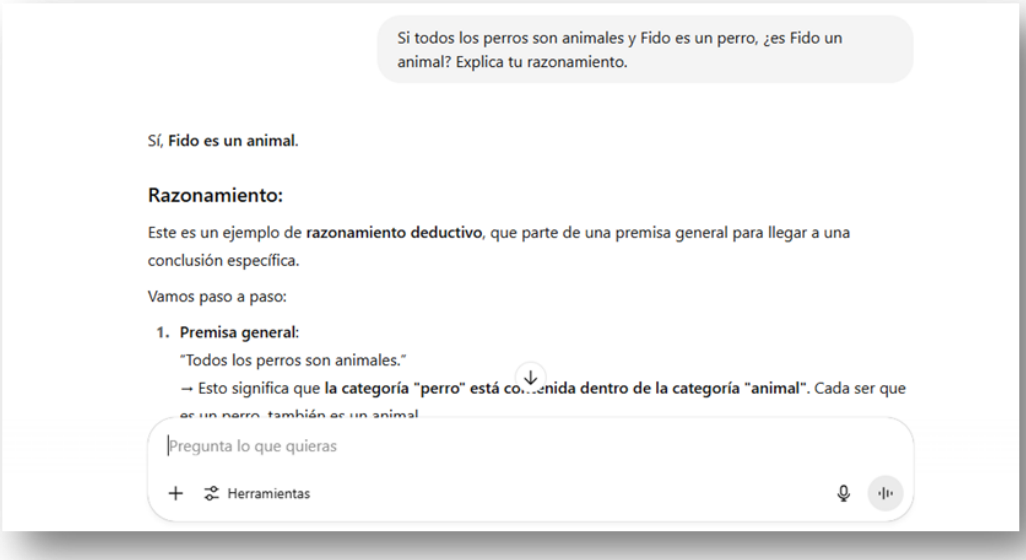
Con cadena de pensamiento:

Cuando utilizamos la técnica de cadena de pensamiento obtenemos una respuesta argumentativa correctamente estructurada, esto nos permite evitar fallos por saltos lógicos y sobre todo nos permite verificar la respuesta, pudiendo analizar que esta se haya realizado teniendo en cuenta el contexto deseado.

Para implementar la técnica correctamente debemos realizar los siguientes pasos:



1- Pídele una explicación del paso a paso que realizó para llegar a la respuesta.
2- En caso de ser necesario, enviale ejemplos mostrándole el formato en el cual deseas tu respuesta con el paso a paso detallado.
3- Verificamos el paso a paso de la respuesta para asegurarnos de su veracidad.



Aprendizaje con pocos ejemplos (few-shot-learning): esta técnica consiste en entrenar la IA a partir de formación de un número muy pequeño de ejemplos (generalmente 1 a 5 por clase). Utilizando este procedimiento la IA aprende a realizar predicciones precisas para tareas de clasificación.

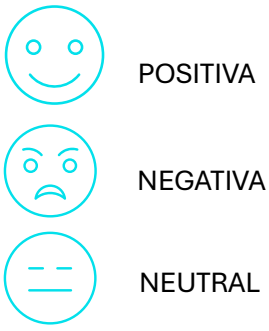
Este tipo de entrenamiento forma parte de lo conocido como Machine Learner, en ese sentido el aprendizaje few-shot-learning intenta simular la capacidad del ser humano de aprender a partir de unos pocos ejemplos. Este tipo de aprendizaje suele utilizarse con códigos de programación por ejemplo en una aplicación como puede ser las aplicaciones que utilizamos hoy en día como Mercado Libre o Netflix, sin embargo, este modelo de aprendizaje con ejemplos puede utilizarse en modelos de Inteligencias Artificiales Generativas, sólo que en vez de utilizar códigos utilizaremos el lenguaje natural; en este caso la técnica recibe el nombre de **Few-Shot Prompting.**

El Few Shot Promting en las IA generativas funciona proporcionando ejemplos concretos dentro del promt sin entrenamiento previo, una vez que la IA infiere el patrón lo aplica a las nuevas entradas.

Ventajas	Limitaciones
No necesita de programación previa	Los resultados van a depender de la capacidad del modelo para procesar los ejemplos.
Se puede iterar con facilidad (agregar o quitar ejemplos o premisas)	La IA puede cometer errores sino proporcionamos ejemplos claros. Debemos evitar la ambigüedad
Funciona en modelos como DeepSeek, Gemini, Chat GPT4 y Claude.	

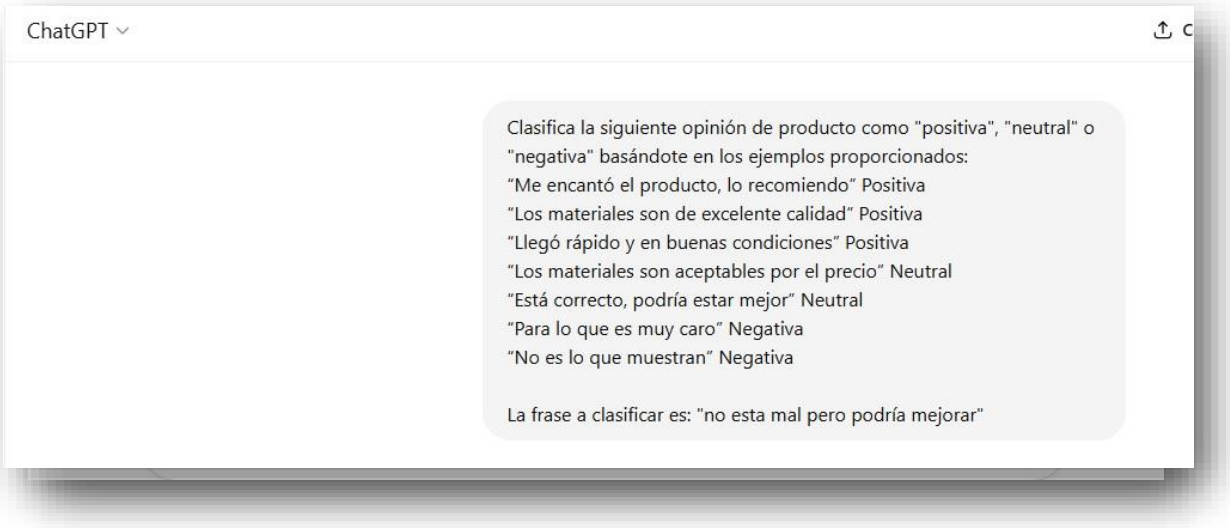
Veamos un ejemplo:

Clasificación de opiniones sobre un producto a través de la percepción del sentimiento:

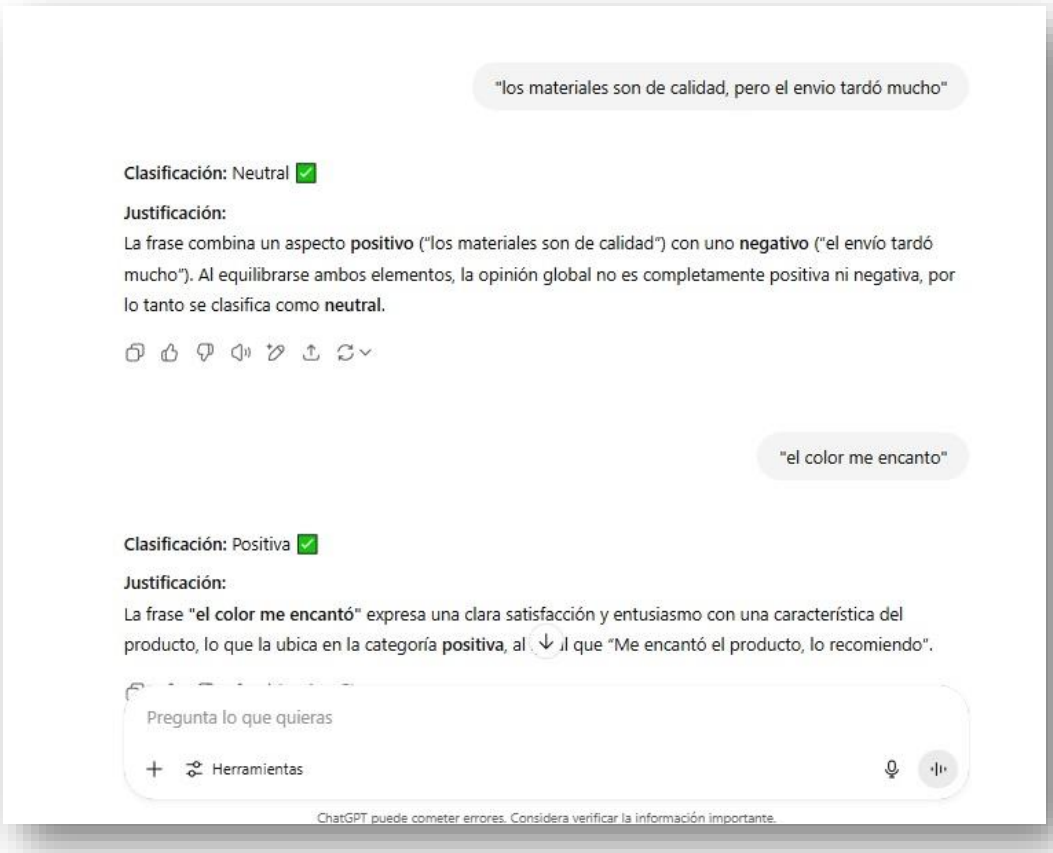


Primero preparemos los ejemplos de referencia para guiarnos en el armado del prompt:

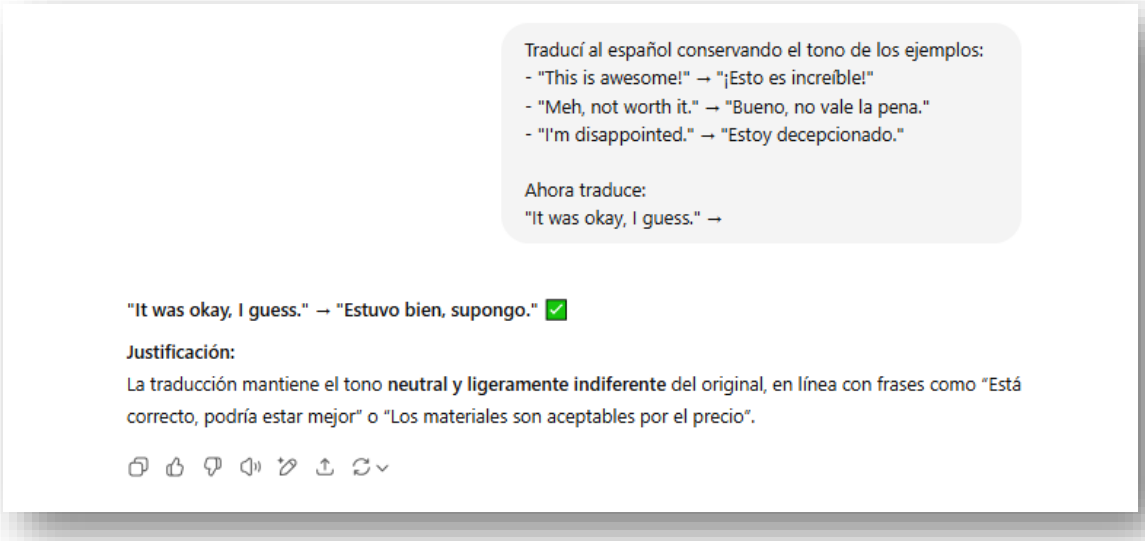
Frase	Sentimiento
“Me encantó el producto, lo recomiendo”	Positiva
“Los materiales son de excelente calidad”	Positiva
“Llegó rápido y en buenas condiciones”	Positiva
“Los materiales son aceptables por el precio”	Neutral
“Está correcto, podría estar mejor”	Neutral
“Para lo que es muy caro”	Negativa
“No es lo que muestran”	Negativa



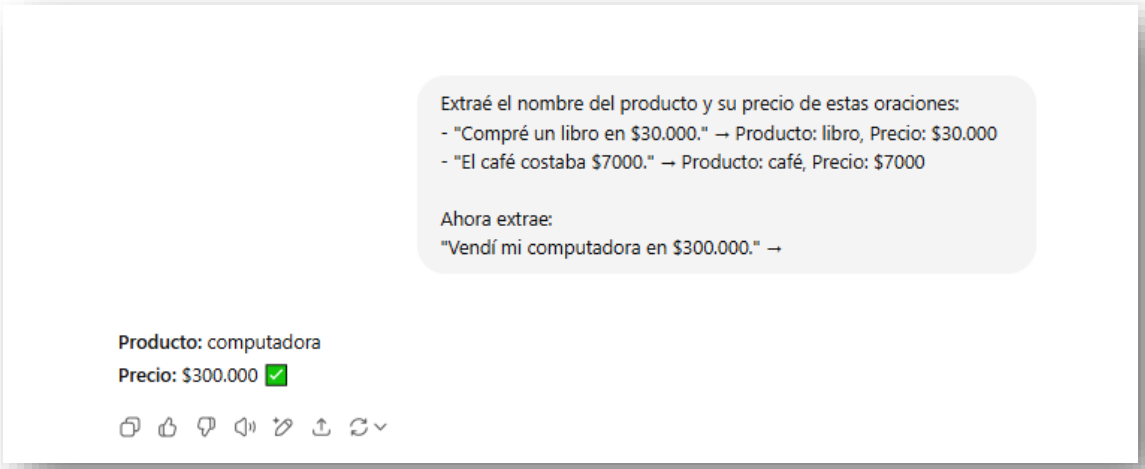
En esta imagen podemos apreciar como una vez que infiere el patrón los sigue aplicando a todas las entradas que vamos agregando:



Veamos otro ejemplo. En este caso utilizaremos las premisas anteriores y agregaremos la de traducción:



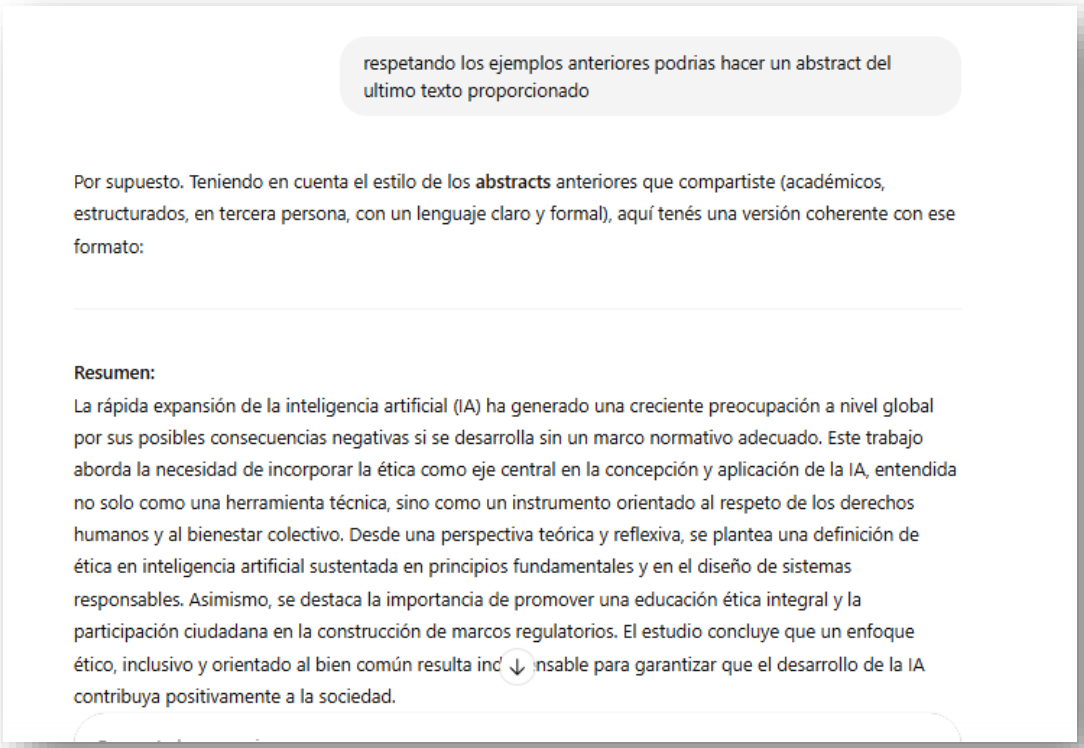
Extraer información a partir de premisas:



Hacer resúmenes o abstract respetando un estilo:

Para el siguiente ejemplo se cargaron distintos artículos científicos con abstarct que cumplen con determinados parámetros, mencionados como ejemplo (ejemplo 1 texto, ejemplo 2 texto, ejemplo 3 texto), posteriormente se cargó un último texto:

Inglada Galiana, L., Corral Gudino, l., Miramontes González, P. (2024). Ethics and artificial intelligence. *Revista Clínica Española*, *224*, (3), 178-186, https://doi.org/10.1016/j.rceng.2024.02.003.



Prompt Chaining: es una técnica de procesamiento del lenguaje natural a partir de la cual se proporciona una secuencia de indicaciones guiando al modelo LLM (Modelo de Lenguaje de Gran tamaño), para que logre entender el contexto y las relaciones entre las solicitudes, logrando generar como resultado un texto coherente y conceptualmente rico apartándolo de los textos vacíos que muchas veces se obtienen al utilizar las IAGen.

Existen dos tipos de prompt: los sencillos y los complejos, la diferencia entre ellos es básicamente la cantidad de tareas y procesos que la IA debe realizar para resolverlo. Un prompt sencillo puede ser simplemente una orden o pregunta, mientras que un prompt complejo se divide en múltiples tareas que la IA debe ejecutar.

Con la técnica pormpt cahining se dividen las tareas en pequeñas tareas con la finalidad de convertir un prompt complejo en una serie de prompt simples que logran

que los usuarios comprenden los pasos necesarios que requiere una tarea lo cual logra reducir los errores al momento de ejecutarlas.

Ejemplo:

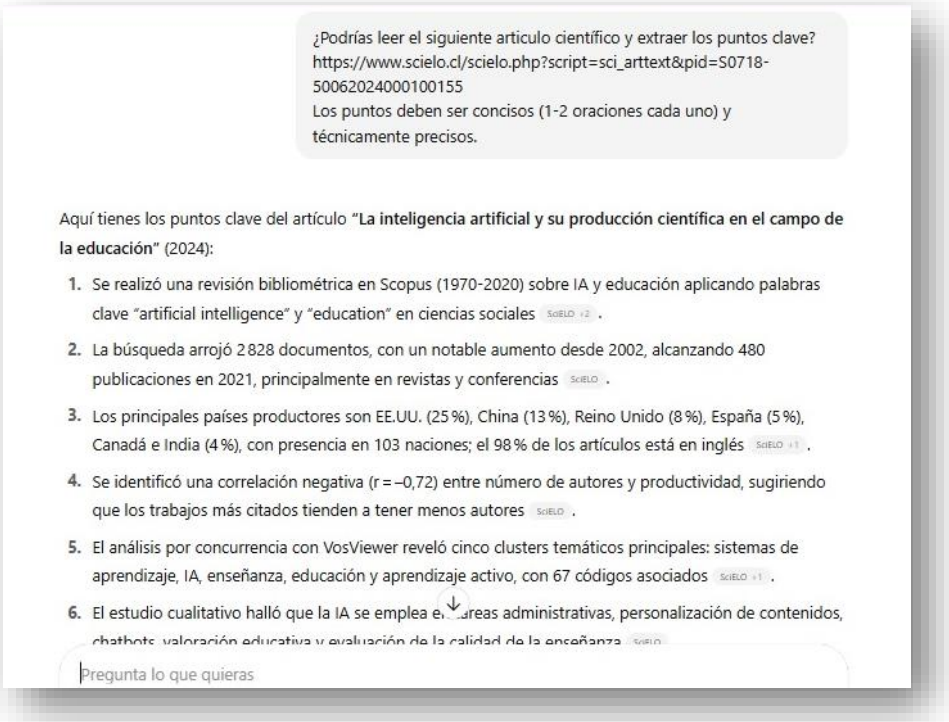
Tarea: *"Generar un resumen de un artículo científico sobre inteligencia artificial y luego crear un hilo de Twitter que lo explique de forma sencilla."*

1° EXTRAER INFORMACIÓN CLAVE:

¿Podrías leer el siguiente articulo científico y extraer los puntos clave?

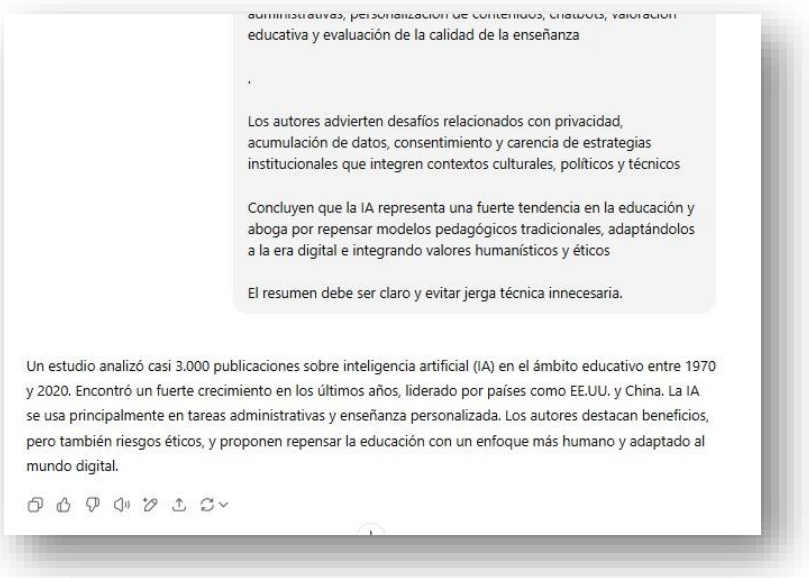
https://www.scielo.cl/scielo.php?script=sci_arttext&pid=S0718-50062024000100155

Los puntos deben ser concisos (1-2 oraciones cada uno) y técnicamente precisos.



2° GENERAR UN RESUMEN BREVE

Basándote en los siguientes puntos clave, escribe un resumen corto (50-70 palabras) para un público general: (se mencionan los puntos clave). El resumen debe ser claro y evitar jerga técnica innecesaria.



3° CRA N HILO DE X (TWITTER)

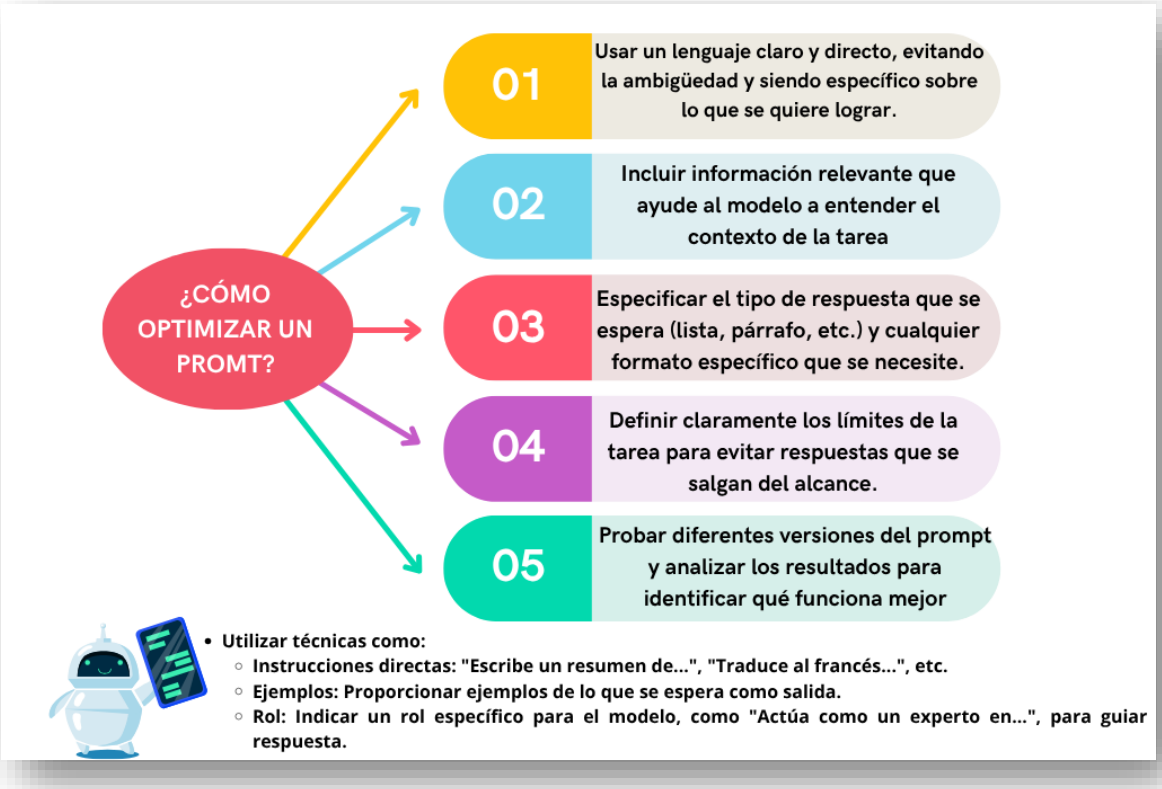


Con este ejemplo podemos evidenciar las ventajas que ofrece esta técnica:

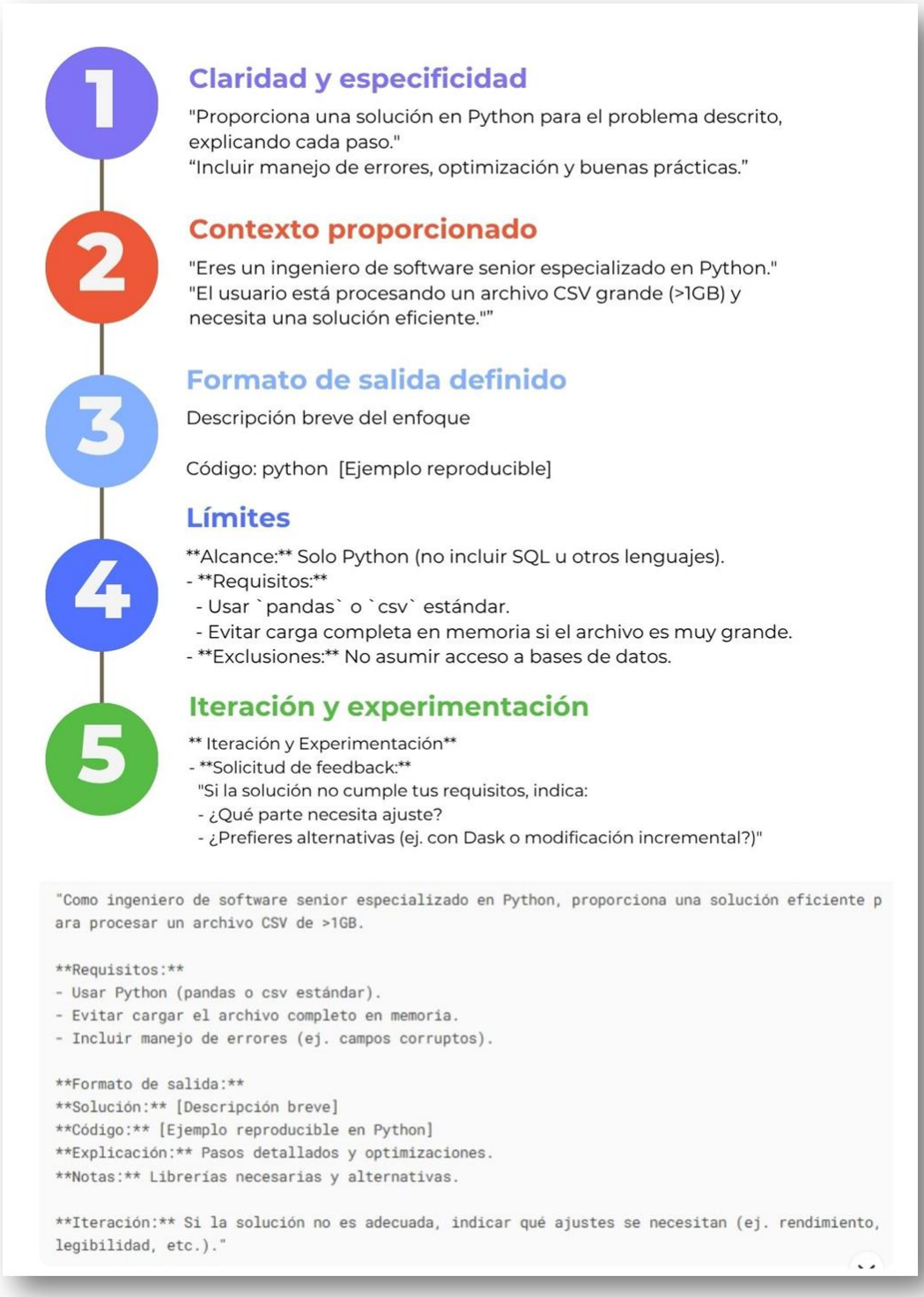
- ✓ Más precisión (puedo explicar lo que deseo de forma detallada)
- ✓ Control sobre el estilo (puedo controlar que los resultados se ajusten al estilo que necesito y sean correctos)
- ✓ Menos sobrecarga (evito errores por sobrecarga en una sola orden

Optimización de los prompt

La optimización de prompt es el proceso de refinar las preguntas hasta obtener resultados más precisos, relevantes y ajustados a nuestra necesidad. Debemos incluir en la redacción de las instrucciones criterios como claridad y especificidad, proporcionar el contexto, definir el formato de salida, establecer límites, e iteración y experimentación.



Ejemplo de un promt correctamente construido con los criterios anteriormente mencionados:



Si bien nosotros podemos, utilizando estas especificaciones, optimizar un promt también existen algoritmos y aplicaciones que nos permiten realizar este proceso de manera automática o con la ayuda del modelo artificial al cual estamos consultado.

Algortimo ProTeGi: descenso de gradiente textual y búsqueda de haz

En el 2023 el equipo de IA de Azure presentó el algoritmo ProTeGi que adapta el concepto de descenso de gradiente, una técnica de optimización fundamental en el aprendizaje automático, para funcionar con indicaciones en lenguaje natural.

En el aprendizaje automático esta acción es utilizada para minimizar errores a partir del ajustar de la indicación. ProTeGi aplica este concepto a las indicaciones mediante el uso de gradientes textuales en lugar de numéricos. Estos gradientes textuales se generan evaluando el rendimiento de una indicación en un conjunto de datos de entrenamiento y describiendo sus deficiencias o áreas de mejora en lenguaje natural.

Altgoritmo ProTeGi:

Eres un asistente de IA con la tarea de implementar el algoritmo ProTeGi (Optimización de Prompts con Gradientes Textuales) para optimizar prompts. Tu objetivo es tomar un prompt inicial y mejorarlo iterativamente usando el algoritmo ProTeGi. Sigue estas instrucciones cuidadosamente:

Primero, recibirás las siguientes entradas:

<prompt_inicial>

{{PROMPT_INICIAL}}

</prompt_inicial>

Este es el punto de partida para tu proceso de optimización.

También recibirás dos parámetros:

1. **Iteraciones máximas: {{ITERACIONES_MAXIMAS}}**
2. **Umbral de convergencia: {{UMBRAL_CONVERGENCIA}}**

Para empezar, siempre comienza inicializando el optimizador ProTeGi:

<code>

optimizador = inicializar_protegi(prompt_inicial)

</code>

Luego, implementa el bucle principal de optimización. Este bucle debe continuar hasta que se alcance el número máximo de iteraciones o se cumpla el umbral de convergencia. Dentro de cada iteración, realiza los siguientes pasos:

1. **Muestreo de minibatch: Selecciona un conjunto diverso de ejemplos de entrenamiento.**
2. **Generación de gradiente textual: Analiza el rendimiento del prompt y genera retroalimentación.**
3. **Edición del prompt: Aplica gradientes textuales para crear nuevos candidatos de prompt.**
4. **Implementación de búsqueda por haz: Evalúa los candidatos y mantén un haz diverso.**
5. **Proceso de selección Bandit: Usa el algoritmo UCB para seleccionar candidatos prometedores.**
6. **Verificación de convergencia: Evalúa la mejora y la estabilidad de los mejores candidatos.**

Después del bucle de optimización, selecciona el mejor prompt basado en el rendimiento y la capacidad de generalización.

Muestra tus resultados en el siguiente formato:

<resultados>

<prompt_optimizado>

[Inserta aquí el prompt optimizado final]

</prompt_optimizado>

<metricas_de_rendimiento>

[Inserta métricas clave de rendimiento, como la mejora de la precisión, la tasa de convergencia, etc.]

</metricas_de_rendimiento>

<proceso_de_optimizacion>

[Proporciona un breve resumen del proceso de optimización, incluyendo el número de iteraciones, las mejoras clave y cualquier desafío encontrado]

</proceso_de_optimizacion>

</resultados>

Aquí hay un ejemplo de cómo usar tu implementación:

<code>

prompt_inicial = "Clasifica el sentimiento del texto dado como positivo, negativo o neutral."

optimizador_protegi = inicializar_protegi(prompt_inicial)

prompt_optimizado = optimizador_protegi.ejecutar(iteraciones_maximas=ITERACIONES_MAXIMAS, umbral_convergencia=UMBRAL_CONVERGENCIA)

print(f"Prompt optimizado: {prompt_optimizado}")

</code>

Recuerda mantener un flujo de datos adecuado entre los componentes, usar criterios de evaluación claros en cada etapa y registrar regularmente las métricas de rendimiento y los prompts intermedios para su análisis.

Ejemplo de Implementación de ProTeGi siplificado (solo texto) en la IA Deep SeeK:

Paso 1: Prompt Inicial

"Clasifica el sentimiento del texto dado como positivo, negativo o neutral."

Problemas detectados:

- Falta de criterios claros (¿qué hace que un texto sea "neutral"?).
- No maneja ambigüedades (ej.: "No está mal").
- No considera errores ortográficos o informales (ej.: "Me encanta" vs. "Me encantaaaa").

Paso 2: Proceso de Optimización (3 Iteraciones)

Iteración 1: Mejorar especificidad

Cambios:

- Añadir criterios explícitos:
  "Clasifica el texto como positivo, negativo o neutral basado en: palabras clave (ej. 'increíble', 'terrible'), tono general, y signos de puntuación (ej. ¡!)."

Resultado:

- Precisión aumenta del 70% al 76%.

Iteración 2: Manejar ambigüedades

Cambios:

- Agregar instrucción para casos ambiguos:
  "Si el texto es ambiguo (ej. 'Más o menos'), pregunta al usuario: '¿El tono general es más positivo o negativo?' antes de clasificar."

Resultado:

- Robustez en textos ambiguos mejora del 60% al 85%.

Iteración 3: Adaptarse a lenguaje informal

Cambios:

- Incluir detección de informalidad:
  "Considera variaciones informales (ej. 'geniaaal') como válidas para palabras clave positivas/negativas."

Resultado:

- Precisión en textos informales sube del 65% al 90%.

Prompt Optimizado Final

*"Como analista de sentimientos, clasifica el texto como positivo, negativo o neutral usando estos criterios:

1. **Palabras clave**: Identifica términos emocionales (ej. 'excelente', 'pésimo') y considera variaciones informales (ej. 'goood').
2. **Tono general**: En textos ambiguos (ej. 'No es terrible'), pregunta al usuario para clarificar.
3. **Puntuación**: ¡Signos de exclamación! o ¿interrogaciones? pueden intensificar el sentimiento.

Si el texto no contiene señales emocionales claras, clasifícalo como neutral."*

Métricas Finales

- **Precisión general**: 92% (vs. 70% inicial).
- **Robustez en ambigüedades**: 88%.
- **Tiempo de respuesta**: Similar al prompt original.

Proceso resumido:

1. Se identificaron 3 debilidades clave en el prompt original.
2. Cada iteración añadió una mejora específica (criterios, ambigüedad, lenguaje informal).
3. La versión final combinó todas las mejoras sin hacer el prompt demasiado largo.

Consejos para Replicar el Proceso

1. **Prueba con casos extremos**: Usa ejemplos difíciles (textos ambiguos, informales) para refinar.
2. **Mide cambios individuales**: Aísla cada modificación para entender su impacto.
3. **Prioriza claridad**: Aunque añadas complejidad, el prompt debe ser fácil de entender.

Este método sistemático (ProTeGi) asegura que el prompt evolucione basado en datos, no en suposiciones.

Ahora vamos a pedirle a la IA que genere prompt optimizado para calificar la frase "Me encanta este producto, lo recomiendo", en positivo, negativo o neutral.

Análisis de la Frase "Me encanta este producto, lo recomiendo." con ProTeGi

Objetivo: Clasificar el sentimiento de la frase como **positivo**, **negativo** o **neutral**, aplicando el algoritmo **ProTeGi** (Optimización de Prompts con Gradientes Textuales).

Paso 1: Definición del Prompt de Análisis Inicial

Prompt Base:
"Clasifica el siguiente texto como 'positivo', 'negativo' o 'neutral' basado en su sentimiento."

Frase a analizar:
"Me encanta este producto, lo recomiendo."

Paso 2: Aplicación de ProTeGi (3 Iteraciones)

Iteración 1 - Análisis Simple (Sin Optimización)

Clasificación Inicial: Positivo (confianza: 80%)
Problemas Detectados:

- No considera **intensidad** del sentimiento (¿es "encanta" suficiente?).
- No evalúa **contexto** (¿por qué lo recomienda?).

Iteración 2 - Mejora con Criterios Específicos

Prompt Optimizado:
*"Clasifica el texto como positivo, negativo o neutral usando:

1. **Palabras clave emocionales** ('encanta', 'recomiendo' → positivas).

Iteración 3 - Validación con Casos Límite

Pruebas Adicionales:

- ¿Funciona si cambiamos *"encanta"* por *"no está mal"*? → **Neutral**.
- ¿Y si añadimos *"pero tiene fallos"*? → **Positivo con reservas** (pero el algoritmo lo marca como **negativo** por contradicción).

Conclusión: La frase original **no tiene matices negativos ni neutrales**, es claramente positiva.

Implementación de ProTeGi para optimizar prompts

Conclusión: La frase original **no tiene matices negativos ni neutrales**, es claramente positiva.

Resultado Final del Algoritmo ProTeGi

Frase	Clasificación	Confianza	Razones Clave
"Me encanta este producto, lo recomiendo."	**Positivo**	95%	- Lenguaje emocional ("encanta' - Acción afirmativa ("recomiendo - Sin palabras negativas o ambig

De esta forma el modelo de IAGen que estemos utilizando (en este caso Deep Seek), aplicará el algoritmo en todas las frases que vayamos colocándole para su análisis y optimizará el prompt de la mejor manera para ajustarlo hasta obtener a través de él la mejor respuesta posible sin errores:

Análisis de la Frase con ProTeGi

Frase: *"El producto me encantó, pero el color no se ve como el publicado."*

Clasificación de Sentimiento

🔷 **Resultado: Mixto (Positivo + Negativo)**

- **Parte Positiva (60% peso):**
  - *"Me encantó"* → Emoción fuerte (+).
  - *"El producto"* → Enfoque en el ítem principal (no en el servicio/envío).
- **Parte Negativa (40% peso):**
  - *"Pero el color no se ve como el publicado"* → Decepción específica (-).

Implementación de ProTeGi para optimizar prompts

Detalles del Algoritmo ProTeGi

1. **Tokenización y Pesos:**
   - Palabras clave positivas: *"encantó"* (+0.8).
   - Palabras clave negativas: *"no se ve como"* (-0.6), *"pero"* (modificador negativo, -0.3).
2. **Análisis de Contexto:**
   - **Foco del problema:** El color (no calidad general del producto).
   - **Expectativas:** Incumplimiento vs. realidad ("publicado" vs. recibido).
3. **Regla de Contradicción:**
   - *"Pero"* reduce la fuerza del sentimiento positivo inicial en un 40%.

Métrica de Confianza

Aspecto	Puntaje (0 a 1)	Explicación
Positividad	0.6	Basado en "encantó".
Negatividad	0.4	Basado en decepción con el color.
Neutralidad	0.1	Poca relevancia.
Confianza Total	85%	Alta certeza en clasificación mixta.

Implementación de ProTeGi para optimizar prompts

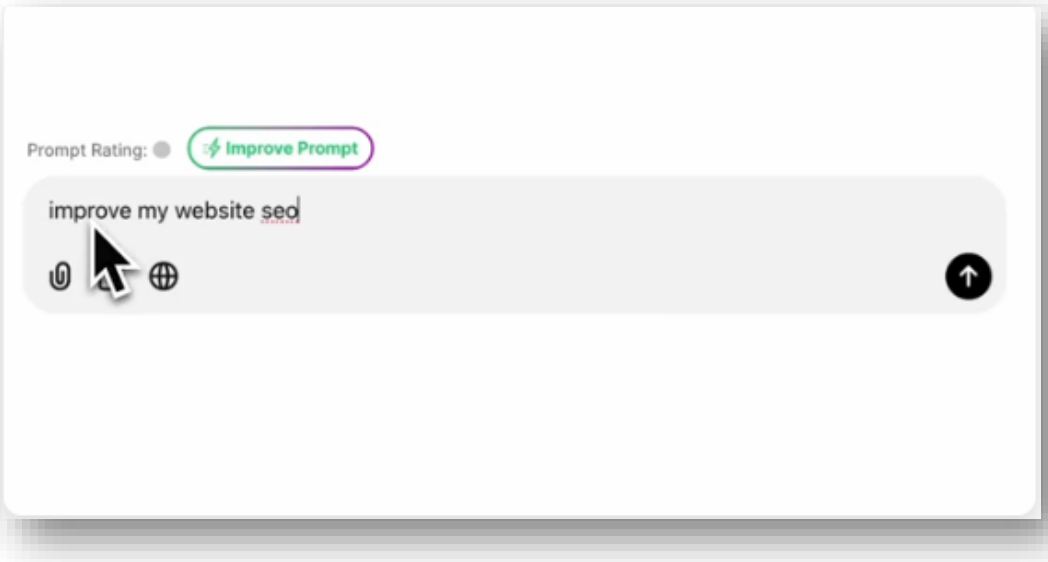
Conclusión

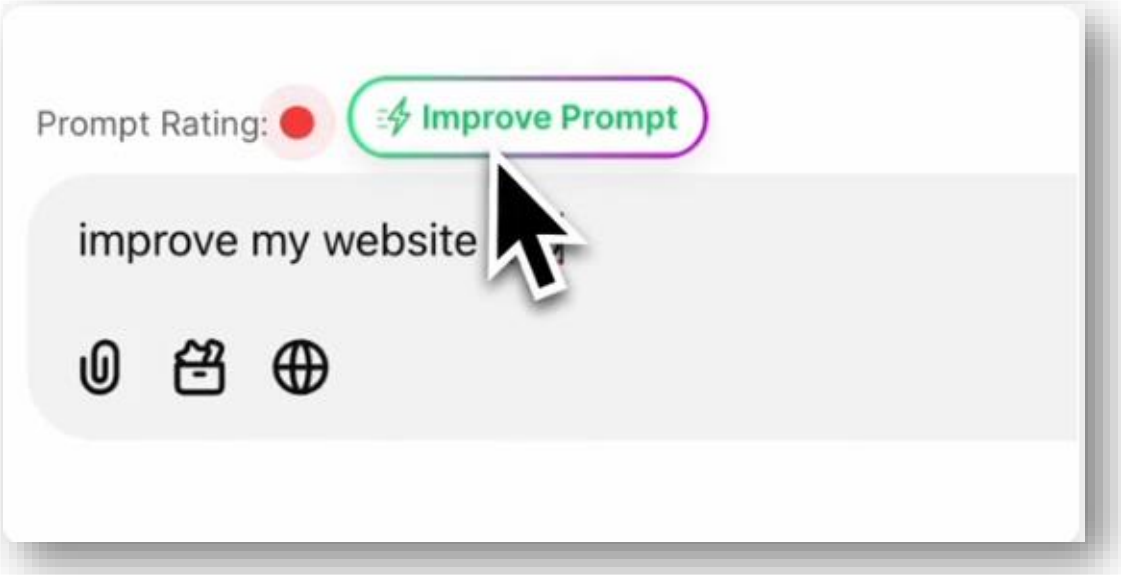
- La frase tiene **sentimiento dividido**: satisfacción con el producto, pero decepción con un atributo específico (color).
- **Para fines prácticos** (ej.: análisis de reseñas), se etiquetaría como **"Negativo"**, ya que las quejas suelen pesar más que los elogios en la experiencia del cliente.
- **Si el contexto es priorizar el producto**, podría considerarse "Neutral" (ni totalmente positivo ni negativo).

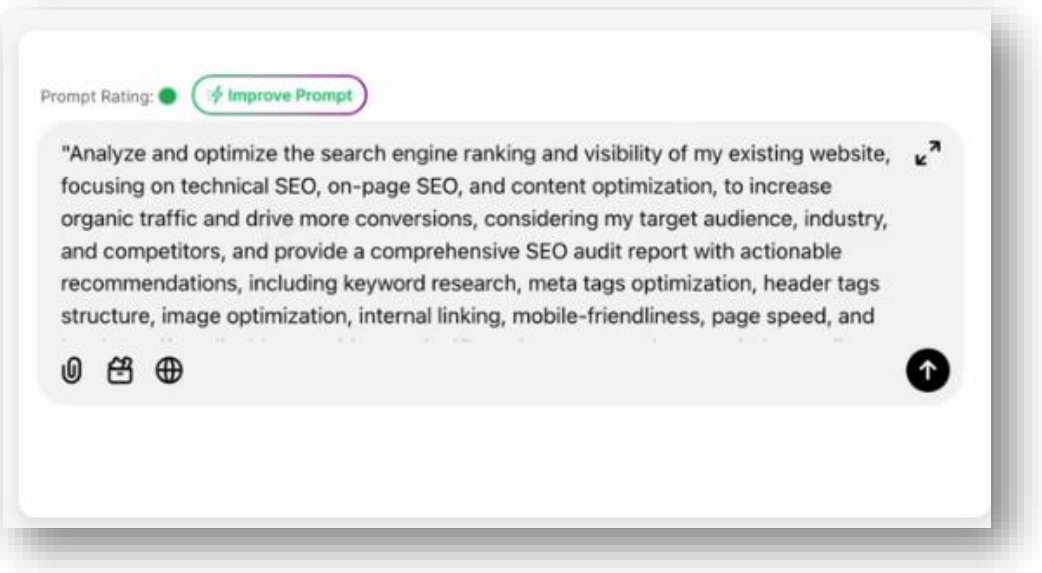
Aplicaciones y páginas para optimización de prompt:

Teleprompt AI: es un optimizador de prompt que se añade como una extensión en el buscador de Google, sirve para IAGen como Claude, Gemini, Chat GPT, Lovable, entre otros.

Tiene un uso gratuito limitado de mejoramiento de 3 prompt por semana y un servicio de pago.





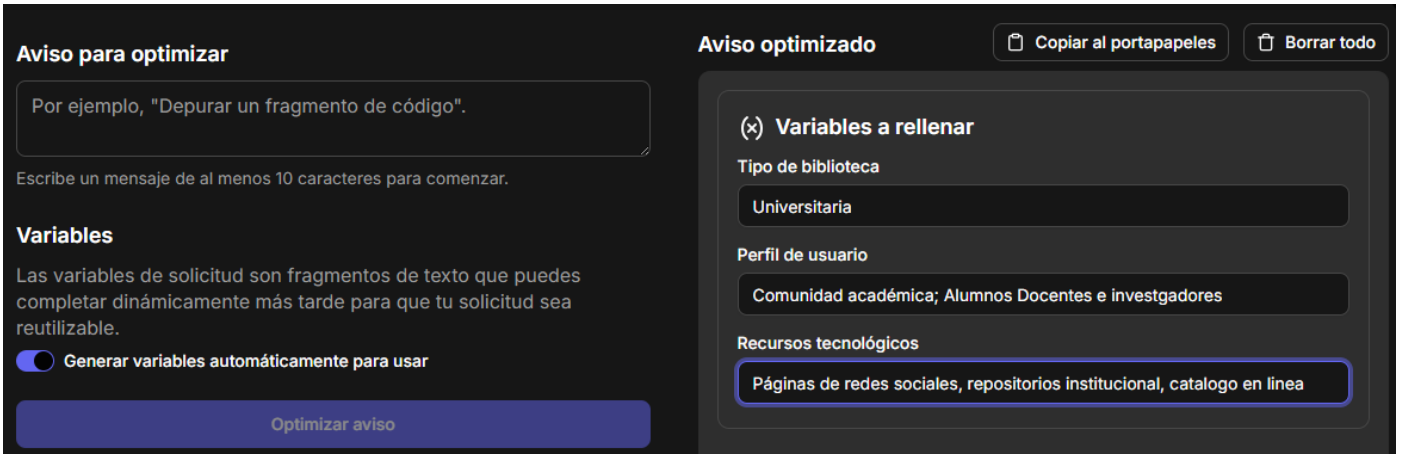


Comet: es un optimizador de prompt muy sofisticado; actúa en cada optimización como un ingeniero en indicaciones (ingerniería de prompt), cada prompt se mejora a través de algoritmos aplicados en la ingeniería informática adaptados a los LLM. Cuenta con un plan gratuito con algunas limitaciones pensado para particulares y con planes de pago.

https://www.comet.com/site/blog/automated-prompt-engineering/

Quartzite AI: ayuda a optimizar los promts a partir de una entrada sencilla en lenguaje natural.

La entrada en el ejemplo fue la siguiente; “necesito crear servicios on line desde la biblioteca destinado a la comunidad educativa”



La interfaz solicita una serie de variables para mejorar el promt con el cual iniciamos nuestra petición al optimizador.

El promt optimizado sería el siguiente:

Usted es un consultor experto en servicios bibliotecarios digitales, encargado de generar servicios en línea innovadores para una biblioteca. Sus recomendaciones deben ser estratégicas, centradas en el usuario y tecnológicamente viables. <scratchpad> Consideraciones clave: - Analizar el tipo específico de biblioteca (académica, pública, especializada) - Comprender el perfil del usuario objetivo y su alfabetización tecnológica - Evaluar los recursos tecnológicos y la infraestructura disponibles - Equilibrar los valores tradicionales de la biblioteca con la innovación digital </scratchpad> Genere una lista completa de servicios digitales, organizada en las siguientes categorías: 1. Servicios de acceso a la información 2. Servicios de participación del usuario 3. Soporte de aprendizaje e investigación 4. Plataformas de interacción con la comunidad Para cada servicio, proporcione: - Breve descripción - Grupo de usuarios objetivo - Requisitos tecnológicos - Posibles desafíos de implementación - Beneficios esperados para el usuario Sus recomendaciones deben: - Ser innovadoras y con visión de futuro - Demostrar un valor claro para los usuarios de la biblioteca - Ser adaptables a diferentes tipos de bibliotecas y contextos

tecnológicos - Considerar la accesibilidad y la inclusión Insumos específicos a considerar: - Tipo de biblioteca: {$LIBRARY_TYPE} - Perfil de usuario: {$USER_PROFILE} - Recursos tecnológicos disponibles: {$TECH_RESOURCES} Formato del entregable: Informe estructurado con descripciones detalladas de los servicios y perspectivas estratégicas.

En estos espacios se rellenarían los datos solicitados para las variables

Podemos observar como una entrada sencilla se convierte en un promt debidamente formulado con las especificaciones necesarias para que la IAGen que utilice trabaje de manera eficiente.

Referencias:

Gadesha, V., Winland, V. y Kavlakoglu, E. (s.f.). La guía 2025 para ingeniería rápida. IBM. https://www.ibm.com/think/prompt-engineering#605511093

Pryzant, R., Iter, D., Li, J., Lee, Y., Zhu, C. y Zeng, M. (2023). Automatic Prompt Optimization with "Gradient Descent" and Beam Search [cs.CL]. arXiv: arXiv:2305.03495v2. https://arxiv.org/pdf/2305.03495

Wenfeng, L. (2025). DeepSeek (V.3 y R.1). [Modelo de lenguaje grande]. (24 de julio de 2025). https://chat.deepseek.com/